

Database Replication

Phase 2 · Intermediate · Estimated time: 1.5 hours

1. Concept From First Principles

Chapter 1 used read replicas as a scaling tool without explaining exactly how the underlying replication mechanism works. This chapter opens that box: replication is the process of keeping copies of the same data synchronized across multiple database nodes, and the specific replication *topology* you choose has real consequences for consistency, availability, and operational complexity that go well beyond "just add more replicas."

Single-leader (primary-replica) replication is the most common pattern: one node is designated the leader and accepts all writes; every other node is a follower that only accepts replicated changes from the leader, never direct writes. The leader records every change to a **replication log** (in PostgreSQL, the write-ahead log; in MySQL, the binlog), and followers continuously read and apply that log to stay in sync. This is simple to reason about — there's exactly one place writes can conflict, which is nowhere, since only one node accepts them — which is why it's the default choice for the vast majority of production systems.

Synchronous vs. asynchronous replication

Within single-leader replication, you additionally choose **how** the leader waits for followers.

Synchronous replication means the leader waits for at least one follower to confirm it has durably stored the write before telling the client the write succeeded — this guarantees that even if the leader crashes the instant after acknowledging, the write survives on the confirmed follower, at the cost of added write latency and reduced availability if that follower is slow or unreachable. **Asynchronous replication** means the leader acknowledges the client immediately and streams the change to followers in the background — faster writes, but a leader crash immediately after acknowledging can lose that write entirely, since it may not have reached any follower yet. Most systems actually run a hybrid: synchronous to one nearby follower for durability, asynchronous to the rest for read-scaling, balancing safety against latency.

Backend engineer analogy

This mirrors the choice between a Laravel `DB::commit()` that waits for confirmation versus firing an event and moving on — you're weighing "How sure do I need to be that this actually landed before I tell the user it worked?" against "How much latency am I willing to add to every request to get that certainty?" Database engineers make this exact call at the replication layer, just with much higher stakes and much more infrastructure involved.

Multi-leader replication allows more than one node to accept writes, typically one leader per data center or region, with leaders replicating changes to each other. This solves the latency problem of forcing every write in, say, a UAE data center to travel to a single leader sitting in an Indian region — but it introduces a genuinely hard new problem: **write conflicts**, where the same record is modified differently on two leaders at nearly the same time, and the system must have a deterministic rule

(last-write-wins by timestamp, custom merge logic, or manual resolution) to reconcile them. Multi-leader setups are noticeably more operationally complex and are usually only justified when write latency across geographic regions is itself a hard business requirement.

Leaderless replication (used by Cassandra, DynamoDB, and Riak) removes the single-leader concept entirely: any node can accept a write, and the system uses **quorum** rules (write to W nodes, read from R nodes, where $W + R$ greater than the total node count N guarantees overlap) to keep reads and writes consistent enough without a designated leader. This trades away the simplicity of single-leader replication for high write availability, even during network partitions — a direct, practical instance of the AP choice from Chapter 4's CAP theorem.

A failure scenario worth understanding precisely, since interviewers probe it often: what happens when the leader in a single-leader setup dies? The system needs a **failover** process (Chapter 2) to promote a follower to the new leader. This is trickier than it sounds — if the old leader had accepted writes that hadn't yet replicated to the newly-promoted follower, those writes are lost, and if the old leader comes back online later still believing it's the leader, you can end up with two nodes both accepting writes simultaneously, a dangerous scenario called **split-brain** that most production systems guard against with fencing tokens or consensus protocols.

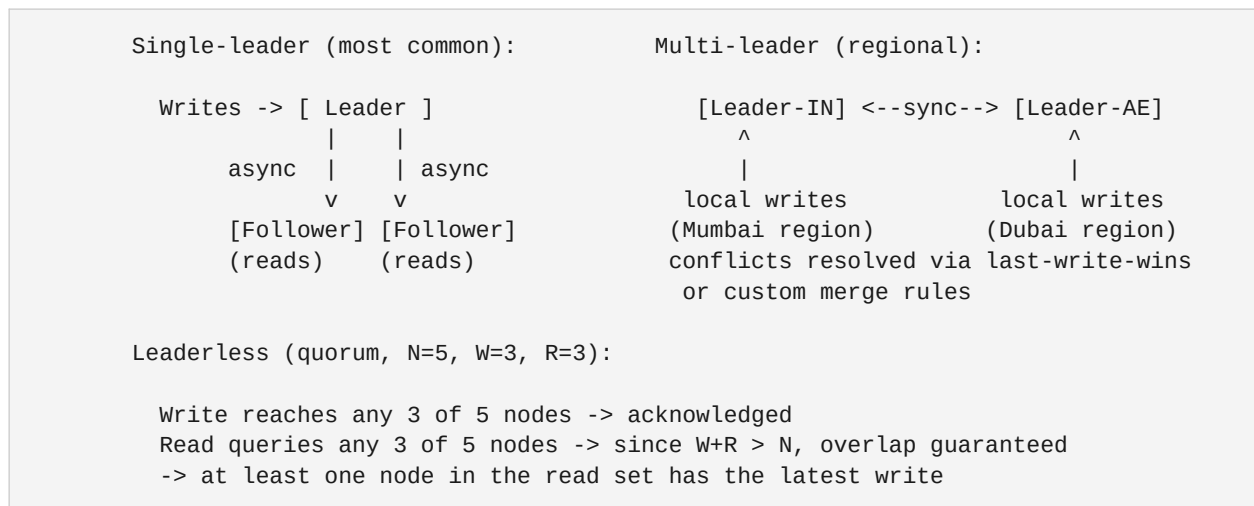
2. Real-World Example / Case Study

Most relational-database-backed systems at Flipkart, Swiggy, and Razorpay scale, run single-leader replication as their default: one primary handling writes, several asynchronous read replicas absorbing read traffic, exactly as described in Chapter 1, with the replication mechanism from this chapter being the machinery underneath that pattern.

A company like Careem or Talabat, operating across multiple Gulf countries with genuinely latency-sensitive local writes (drivers updating their location every few seconds), might consider multi-leader replication with one leader per region — accepting the added conflict-resolution complexity in exchange for local write latency that a single, distant leader could never provide.

Global-scale systems like DynamoDB (used internally at many large platforms, and directly by some fintechs like STC Pay's infrastructure partners) lean on leaderless, quorum-based replication specifically because it can keep accepting writes even when entire data centers become unreachable — a deliberate AP choice appropriate for high-volume, less strictly-ordered data like session state or activity logs, though notably not for the core ledger itself, which typically still demands the stronger guarantees of a single-leader or consensus-based system.

3. Diagram



4. Trade-offs Table

Topology	Strength	Weakness
Single-leader, async replicas	Simple, no write conflicts, good read scaling	Replication lag; leader is a write bottleneck
Single-leader, sync replica(s)	Strong durability guarantee on writes	Higher write latency; availability tied to follower health
Multi-leader	Low local write latency across regions	Write conflicts must be resolved; complex to operate
Leaderless (quorum-based)	High write availability, partition-tolerant	Eventual consistency; tunable but never as simple as single-leader

5. Common Interview Questions

Q1: "Explain single-leader replication and why most systems default to it."

Model approach: Describe the one-writer, many-readers structure, then explain that funneling all writes through one node eliminates write conflicts entirely by construction, which is why it's the simplest, safest default — and only reach for multi-leader or leaderless when single-leader's constraints (write throughput, cross-region write latency) become a proven, measured bottleneck.

Q2: "What happens if the leader fails, and what could go wrong during recovery?"

Model approach: Explain the failover process — a follower is promoted to leader — and name the two concrete risks: unreplicated writes on the dead leader being lost, and split-brain if the old leader comes back believing it's still in charge, mentioning fencing tokens or a consensus mechanism as the fix.

Q3: "When would you choose multi-leader replication despite its complexity?"

Model approach: Name the specific trigger — genuinely latency-sensitive local writes across distant geographic regions where a single, distant leader would add unacceptable round-trip latency — and acknowledge explicitly that you're taking on write-conflict resolution as a real cost in exchange.

Q4: "Synchronous or asynchronous replication for a payments database, and why?"

Model approach: Synchronous, at least to one nearby follower — justify by explaining that losing an acknowledged payment write because the leader crashed before replicating it asynchronously is not an acceptable risk for financial data, even though it costs some write latency.

6. Practice Exercises

- 1 Draw the failover sequence when a single leader crashes: what the system must detect, decide, and do, step by step, and name one thing that can go wrong at each step.
- 2 A ride-hailing company wants sub-50ms write latency for driver location updates in both India and UAE. Propose a replication topology and justify the trade-off you're accepting.
- 3 Explain why leaderless (quorum) replication is described as 'eventually consistent' even though reads and writes both touch a majority of nodes.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Database Replication Explained: Master-Slave vs Master-Master Patterns"	Clear comparison of single-leader vs multi-leader topologies.
English	"MySQL Database Replication Master-to-Slave"	Concrete, hands-on look at how binlog-based replication actually works.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers replication topologies as part of a full bilingual system design course.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Places replication in context of the broader Hindi HLD roadmap.

8. Key Takeaways

- Single-leader replication funnels all writes through one node, eliminating write conflicts by design.
- Synchronous replication trades write latency for durability; asynchronous trades durability for speed.
- Multi-leader replication enables low local write latency across regions but introduces write conflicts.
- Leaderless (quorum-based) replication uses $W+R > N$ overlap rules for high write availability.
- Leader failover is genuinely risky: unreplicated writes can be lost, and split-brain is a real failure mode.
- Default to single-leader unless a specific, measured need (cross-region write latency, extreme write availability) justifies the added complexity of multi-leader or leaderless.
- Replication topology choice is a direct, practical instance of the CAP/PACELC trade-offs from Chapter 4.